

RN2384 hook-up guide (on Cibicom/Loriot server)

- using ESP32

Setting up the ESP32

1. Install the RN2483 Library: <https://github.com/jpmeijers/RN2483-Arduino-Library> in your Arduino IDE. If unsure how to install a library use instructions here: <https://www.arduino.cc/en/Guide/Libraries>
2. Make the following connections between your ESP32 and RN2483 module. Its recommended to place both modules in a breadboard.

ESP32 **GPIO19** <-----> **RX** on RN2483

ESP32 **GPIO18** <-----> **TX** on RN2483

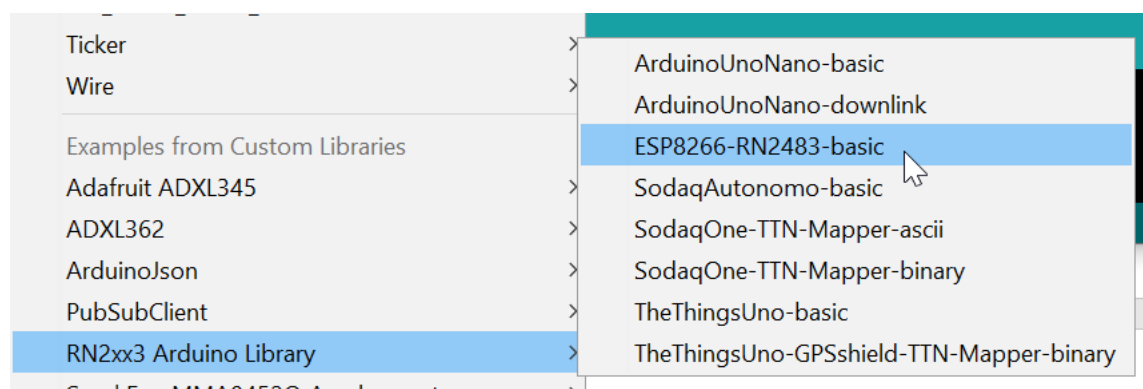
ESP32 **3.3V** <-----> **3.3V** on RN2483

ESP32 **GND** <-----> **GND** on RN2483

3. ESP32 **GPIO23** <-----> **RST** on RN2483

Note: Look up the ESP32 pinout if unsure which pins are GPIO19, GPIO18 and GPIO23. Also, the UART pins chosen here (GPIO19 for TX, GPIO18 for RX) are just a suggestion. You may change these if you want or need to.

4. Connect the antenna to the RN2483 board.
5. In the Arduino IDE, load example code *Examples -> RN2xx3 Arduino Library -> ESP8266-RN2483-basic*



Note that this example code is made for joining and transmitting to a TTN network (The Things Network).

6. Inspect the code and analyse what it does.
7. Now make changes to reflect that you are using a ESP32 and not a ESP8266. This would include to replace the SoftwareSerial with a HardwareSerial. Much like it was done in the Lora P2P exercise. Maybe some inspiration can be found in the screenshots below. Remember to comment, document and describe the code you use in the PDF report.

```
//  
#include <rn2xx3.h>           // Needed for certain RN2483 commands  
//#include <SoftwareSerial.h>  
#include <Arduino.h>         // Makes available a number of the usual Arduino references  
#include <HardwareSerial.h>   // Needed to initialize additional hardware UARTs  
  
HardwareSerial mySerial(1);    // We're going to use UART1  
  
#define RXD2 18  
#define TXD2 19  
#define RST 23  
  
#define RESET 23              // GPIO23 connected to RN2483 RESET pin (RST)  
//SoftwareSerial mySerial(4, 5); // RX, TX !! labels on relay board is swapped !!
```

```
.....  
  
//create an instance of the rn2xx3 library,  
//giving the software UART as stream to use,  
//and using LoRa WAN  
rn2xx3 myLora(mySerial);  
  
// the setup routine runs once when you press reset:  
void setup() {  
    // LED pin is GPIO2 which is the ESP8266's built in LED  
    pinMode(2, OUTPUT);  
    led_on();  
  
    // Open serial communications and wait for port to open:  
    //mySerial.begin(57600);  
  
    Serial.begin(57600); // Serial communication to PC  
    mySerial.setRxBufferSize(1024); //  
    mySerial.begin(57600, SERIAL_8N1, RXD2, TXD2); // Initialize UART communication to RN2483  
    mySerial.setTimeout(1000); // Set timeout (in ms) to wait for serial data  
  
    delay(1000); //wait for the arduino ide's serial console to open
```

Before uploading the sketch make sure that you have connected the antenna.

Upload and run the code and open the serial monitor. Remember to set the correct baud rate. You will find this in the code in Arduino IDE.

If you see an error message *“Communication with RN2xx3 unsuccessful. Power cycle the board.”* double-check the connections. Also try to push the RST button on the ESP8266 module.

8. If the setup and connection is successful you should see something like this in the serial monitor:

```
Communication with RN2xx3 unsuccessful. Power cycle the board.  
ct 31 2018 15:06:52  
When using OTAA, register this DevEUI:  
0004A30B00F2258B  
RN2xx3 firmware version:  
RN2483 1.0.5 Oct 31 2018 15:06:52  
Trying to join TTN  
Unable to join. Are your keys correct, and do you have TTN coverage?
```

From this output you will see the code and library have retrieved the hardware unique EUI number for the RN2483 module (DevEUI). This is a 16 hexadecimal characters identification number that we will need to add the device to the LoraWAN server (Cibicom server). The DevEUI was retrieved using this call

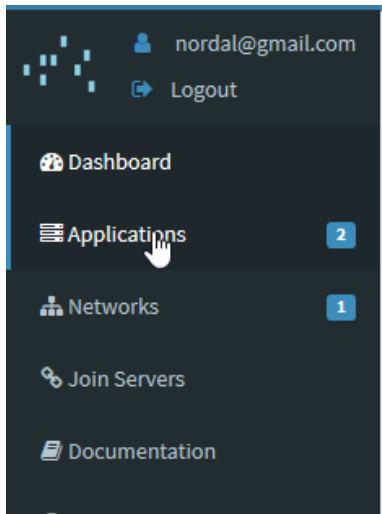
```
hweui = myLora.hweui();
```

Copy-paste the EUI number to a Notepad or similar. You will need it soon.

Note: You will likely also see a message that it was unable to join. This is because we have not inserted proper join credentials yet.

Joining the RN2483 to the Cibicom/Loriot network

- Log into the Cibicom / Loriot server with the credentials you have received (<https://iotnet.teracom.dk/>) and click “Applications”

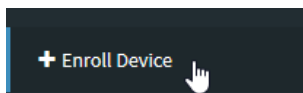


In Applications, use the existing application created called *IoT-course*. Notice the max devices allowed per application. This cannot be exceeded.

Network Applications

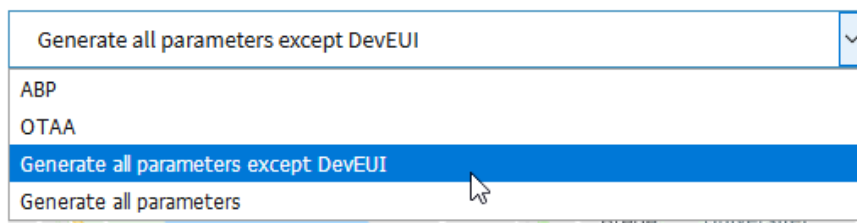
Applications				
	Name	App Id	Devices	Max. Devices
	Filter by name			
☐	IoT-course	BE-7A-11-09	1	10

- Click on the application and look for “Enroll device” in the left navigation bar. Click the link.



- Now you can enrol your RN2483. Select the “Enrollment process”, in this case we will use Over the Air (OTAA) but we will let Loriot / Cibicom create the necessary Application EUI and Application Key. This is done by selecting “Generate all parameters except DevEUI”:

Enrollment process



Now insert the DevEUI that you got from the device before (HWEUI) and click Enroll:

Device Details

DevEUI

0004A30B0021BF-B3

☐ Create Another

Enroll

12. Now select your newly enrolled device in the device list:

☐	Device EUI	Name	RSSI (dBm)	SNR (dB)	devSNR (dB)	SF	BAT
☐	00-04-A3-0B-00-21-BF-B3	00-04-A3-0B-00-21-BF-B3			N/A		N/A

From here we can get the Device address (DevAddr), which we don't need now, and the needed Join EUI (JoinEUI). *Note that in the screenshots below JoinEUI is named AppEUI. This is due to a recent name change in the backend – it has no practical meaning. JoinEUI = AppEUI.*

mnpe@fotonik.d... Logout

Back To Devices

DEVICE
00-04-A3-0B-00-21-5B-73

Location

LoRaWAN Parameters

Device Setup Guides

Device Details / 0004A30B00215B73

Device 0004A30B00215B73

Name	00-04-A3-0B-00-21-5B-73
Description	No description available
EUI	0004A30B00215B73 big endian (use by default) 735B21000BA30400 little endian (for LoRaWAN non-compliant devices)
AppEUI	BE7A000000001109 big endian (use by default) 0911000000007ABE little endian (for LoRaWAN non-compliant devices)

AppKey you get from "LoRaWAN parameters" in the Device menu:

mnpe@fotonik.d... Logout

Back To Devices

DEVICE
00-04-A3-0B-00-21-5B-73

LoRaWAN Parameters

LoRaWAN AES128 Keys

AppKey Application Key (Device Key)	... 6AFFC93B7B Remove appkey
--	---------------------------------

If you want to enable over-the-air join, add or derive the device's application

Only AppKey and AppEUI is needed for OTAA join procedure. For ABP (Activation by personalization) join procedure you need Device address (DevAddr) as well as AppSKey and NwkSKey. Note that these are security sensitive session keys. Therefore ABP is less secure than OTAA. Again, in this case we join the network using OTAA so we will not use the AppSKey and NwkSKey session keys. When using OTAA the sessions keys are generated on the device upon joining the network.

13. Now returning to the Arduino IDE. In the code we have the option of using either OTAA or ABP. You will need to un-comment the OTAA line and insert the AppEUI and AppKey you got from the LoraWAN server when adding the device.

```
//ABP: initABP(String addr, String AppSKey, String NwkSKey);  
//join_result = myLora.initABP("02017201", "8D7FFEF938589D95AAD928C2F8  
  
//OTAA: initOTAA(String AppEUI, String AppKey);  
join_result = myLora.initOTAA("70B3D57ED00001A6", "A23C96EE13804963F8
```

14. Deciding what to transmit. In the Arduino IDE, look for this code:

```
Serial.println("TXing");  
myLora.tx("!"); //one byte, blocking function
```

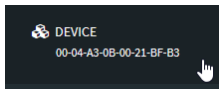
In this case we are transmitting the "!" character. You can change this into something else that you would like to send to the server.

15. In the loop function, change the delay between each transmission. It is set to 200ms which is too frequent. Change this to 2000ms.

```
void loop() {  
    led_on();  
  
    Serial.println("TXing");  
    myLora.tx("!"); //one byte, blocking function  
  
    led_off();  
    delay(200);
```

16. Now upload the code again, open the serial monitor, and this time you should see a message that the device has successfully joined the network, and finally that it is transmitting. It will transmit your message every 2 seconds.

Note: If you are unable to join the network it may be either a) wrong AppKey and AppEUI or b) You do not have Cibicom LoraWAN network coverage where you are. Try a different location. If possible close to DTU where there is coverage.

17. In the Lorient page, click the Device  to see an overview. Here you can see "Last radio" info:

Last radio	
Last seen	Mar 2, 2019, 7:08:54 PM
RSSI	-83.00 dBm
SNR	-21.00 dB
Device SNR	N/A
Frequency	868.300 MHz
SF	12
Bandwidth	125 kHz
Gateway	7076FFFFFF019F0F

And also “Last data” which is the payload we sent:

Last data (10 latest records)			
SeqNo	Time	Port	Data
5	a minute ago	1	0e

Congratulations, you have now transmitted data over a LoraWAN network!

18. The data seen above in “Last data” is only for your device. The device however is sharing application with other devices as well. The data received from all devices can be seen if you go back and click on **Websocket Applications** in the menu and then on the WebSocket sample (see below):



WebSocket sample by LORIoT

Sample application that displays history of data in a table. You can send

<https://iotnet.teracom.dk/application/BE7A1296/websocket>

Click this link to listen to all the data sent to this Application. Notice that you will see data from other devices using the same Application. It will look something like this:

IoT course Websocket

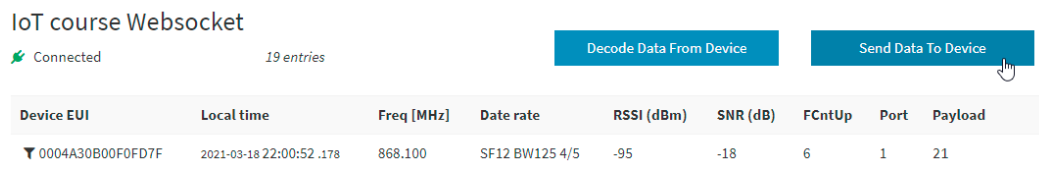
Connected 19 entries Decode Data From Device Send Data To Device

Device EUI	Local time	Freq [MHz]	Date rate	RSSI (dBm)	SNR (dB)	FCntUp	Port	Payload
0004A30B00F0FD7F	2021-03-18 22:00:52 .178	868.100	SF12 BW125 4/5	-95	-18	6	1	21
0004A30B00F0FD7F	2021-03-18 22:00:46 .746	867.100	SF12 BW125 4/5	-98	-14	5	1	21
0004A30B00F0FD7F	2021-03-18 22:00:41 .320	867.700	SF12 BW125 4/5	-97	-4	4	1	21
0004A30B00F0FD7F	2021-03-18 22:00:35 .918	867.500	SF12 BW125 4/5	-111	-21	3	1	21

Notice now you can see which spreading factor (SF), received signal strength (RSSI) and frequency that was used for the transmissions.

Sending data downlink to the RN2483 from Cibicom/Loriot server

19. From this interface we can also send data to the RN2483. This can be done by clicking on “Send data to device” and copy past in the Device EUI you want to send data to, type some data and click send. Choose any port between 1 and 255.



The screenshot shows the 'IoT course Websocket' interface. At the top, it says 'Connected' with a green checkmark and '19 entries'. There are two buttons: 'Decode Data From Device' and 'Send Data To Device'. Below these is a table with the following data:

Device EUI	Local time	Freq [MHz]	Date rate	RSSI (dBm)	SNR (dB)	FCntUp	Port	Payload
0004A30B00F0FD7F	2021-03-18 22:00:52.178	868.100	SF12 BW125 4/5	-95	-18	6	1	21

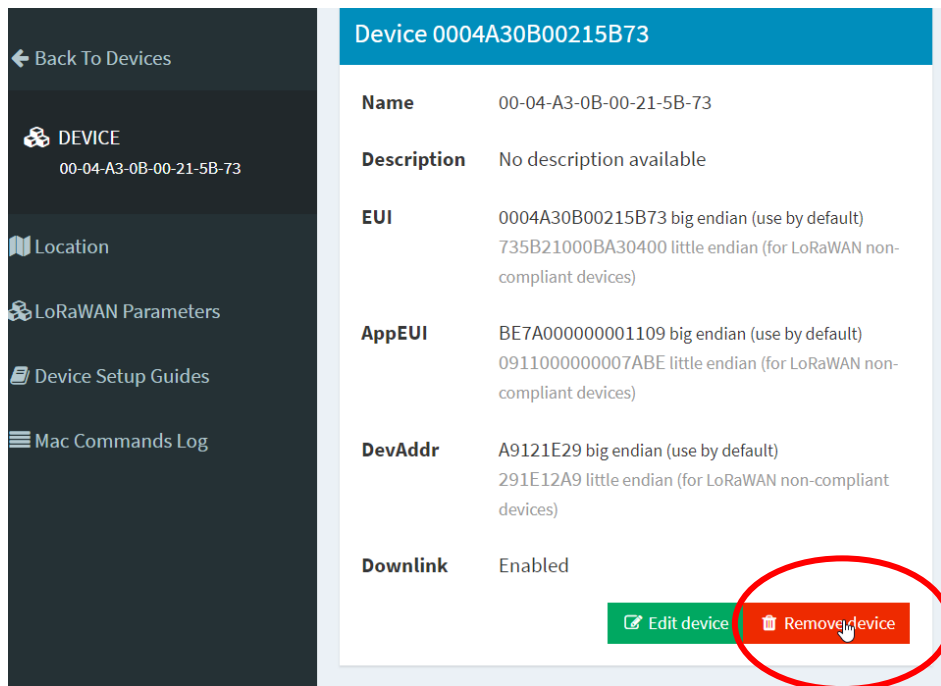
20. Notice how the data gets “queued for sending”. The data is not sent until the RN2483 device transmits and thereby opens up for the one the RX windows.
21. To view the data sent to the device (RN2483) you need to tell the ESP8266 to fetch the received data from the RN2483. The default sketch code does not provide for this. To fetch the data, use the `rn2xx3.h` library function `getRx()`. If your instance is called `myLora`, use `str = myLora.getRx()`; to load the received data into the string, `str`. Then print `str` to the serial monitor.

Reporting the exercise / Assignment:

Although is a guided exercise you can document the process and how you completed the exercise. Describe in your own words and add screenshots and commented code where relevant.

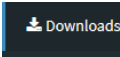
Final Important thing to do!

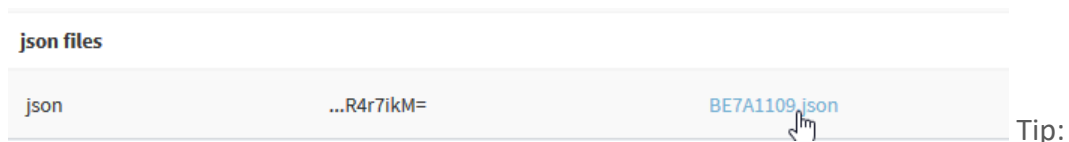
22. Final thing to do is to go and **remove your device** from the Device list. Otherwise the device is locked to this server and application and basically “bricked”. If you want to try out some of the final notes below, wait to remove the device, but please remember to do it when you are done.



Some final notes

- The Websocket application used here is for demonstration. There is no privacy and everyone with the right link can view the data.
- Application output can be chosen as needed. Azure, IBM Cloud, Google IoT, AllThingsTalk and more are available.
- Lots of information about each packet delivered to the server can be found and this can be useful. RSSI (received signal strength), SNR (signal to noise ratio), spreading factor, frequency, which gateways received the package (GW IDs), gateway time stamp, gateway locations (long, lat), sequence number and more. This data is extracted by the loriot server and is based on gateway/radio information derived from the package transmission. This formation can be retrieved through the

Application output. It can also be seen from the  section where you can fetch a .json file including this information about the past received packages.



Open the .json in Firefox as this browser formats .json very nice and readable.

- Every time you power off and on the device (RN2483) you need to re-join the network

About duty cycle and RN2483, and how to by-pass it [only relevant with TTL/FTDI adapter available]

I'm getting a `no_free_ch` response from the module. How can I eliminate it?

This is entirely a duty cycle issue, which is regulatory. Generally, the duty cycle limit is 1% (but it does vary sometimes by frequency band and region) and RN2483 keeps "on air" and "off air" timers for each channel to help ensure that you don't violate the regulations. By default, there are three channels enabled, each with 0.33% duty cycle allocation. When pseudo-randomly hopping around the channels, the algorithm looks at the next channel and decides if the timers have enough remaining duty cycle allocation for the transmission. If not, then it skips that channel and moves to the next. If you've used all the allocation for all channels, you get the `no_free_ch` response. This is quite a common problem during development/learning/debugging, where you tend to send messages far more frequently than normal. In real "sleepy" applications it is rarely an issue. The `no_free_ch` means that all channels are busy transmitting and you have to wait until you can transmit again.

You can cheat the safeguards just by raising the duty cycle limit for each channel using the command `mac set ch dcycle` (from the "**RN2483 LoRa® Technology Module Command Reference User's Guide**", which could be found on the RN2483 product page under the "Documents" section) (<https://www.microchip.com/wwwproducts/en/RN2483>)

The value that needs to be configured can be obtained from the actual duty cycle X (in percentage) using the following formula: $= (100/X) - 1$

- `mac set ch dcycle 0 9` sets channel 0 to 10% (= 90% off)
- `mac set ch dcycle 1 99` sets channel 1 to 1% (= 99% off)
- `mac set ch dcycle 2 999` sets channel 2 to 0.1% (= 99.9% off).

Sending ACK/confirmed messages and receiving confirmations.

Send **un-confirmed**:

`mac tx uncnf 1 [HEX Data]`

Answer to **un-confirmed** messages is always `mac_tx_ok`. This is not an ACK that they message has arrived at the server but just a notice that the message has been sent off.

Send **confirmed** messages (receive ACK that the message has arrived):

`mac tx cnf 1 [HEX Data]`

Answer to **confirmed** messages is `mac_tx_ok` (means message has arrived) or `mac_err` (message did not arrive, or ACK didn't get back to device). I.e. the answer (`mac_tx_ok`) really only makes sense when `mac tx cnf` is used.

Again, remember to go and remove your device from the Device list. Otherwise, the device is locked to this server and application and “bricked”.

Device 0004A30B00215B73

Name	00-04-A3-0B-00-21-5B-73
Description	No description available
EUI	0004A30B00215B73 big endian (use by default) 735B21000BA30400 little endian (for LoRaWAN non-compliant devices)
AppEUI	BE7A000000001109 big endian (use by default) 0911000000007ABE little endian (for LoRaWAN non-compliant devices)
DevAddr	A9121E29 big endian (use by default) 291E12A9 little endian (for LoRaWAN non-compliant devices)
Downlink	Enabled

[Edit device](#) [Remove device](#)

On the device run the “*sys factoryRESET*” command to reset the device for the next user.